

CANDY DIALOGUE ENGINE

TTS

1.0.0

TTS Integration

Candy DE is designed to be compatible with a Text-To-Speech (TTS) tool, to automatically voice dialogues.

To clarify, Candy DE does not provide a built-in TTS tool: you must integrate one on your own.

Such integration can consist of a TTS tool built into your game, a third-party local software, or connection to an online TTS API -- Candy DE doesn't impose any particular requirements in that regard.

That being said, your chosen TTS tool must produce an audio file or an `AudioStream` that can be played in an `AudioStreamPlayer` when prompted.

If your TTS tool outputs a `PackedByteArray` or any other format not directly playable in an `AudioStreamPlayer`, you would have to convert it to an audio file or `AudioStream` before providing it to Candy DE.

TTS Support

TTS can be enabled in `Candy_Settings.gd`, by setting the `text_to_speech` variable to either "Simple" or "Advanced" (or other values if you develop your own TTS logic later on).

Simple mode lets TTS read the same dialogue lines that are displayed on screen. Candy DE ensures that before the lines are sent to the TTS system for reading, any variables are replaced by their values, words are substituted (see the Word Substitution guide, if you use that feature), and BBCode tags are removed to avoid interfering.

Advanced mode instead makes TTS read from a special 'TTS' variant of each line, which can be specifically written with special symbols, tags or spelling to ensure optimal pronunciation, rhythm and tone. This works with any variant at all, including other languages, variants based on speaker or player gender, and even randomized alternate variants (see the Variants & Translations guide for details).

Such 'TTS' variants must be named exactly like the original variant they apply to, with the added "_TTS" suffix at the end. For example:

"Default" → "Default_TTS"

`"French_S:F" → "French_S:F_TTS"`

`"Default_P:M_#4" → "Default_P:M_#4_TTS"`

If you've selected Advanced mode, you can designate a fallback mode in case a line doesn't have the required TTS variant. By default, the fallback is set to Simple mode (see the `tts_advanced_fallback` variable). This will make TTS read the same variant displayed on screen.

If you're concerned that Simple mode would provide poor results as a fallback, you can set the fallback mode to "Off", which will skip TTS entirely. Alternately, you could code your own custom mode as well.

If TTS is enabled in any mode, and if it fails to produce an output for any reason, Candy DE won't crash: it simply won't play any TTS audio.

If TTS is enabled and a line already has a voice file attached, Candy DE will prioritize TTS, and will default to the line's voice file if TTS fails to produce an output.

You can further assign custom TTS settings to actors and to specific lines:

For actors, open `Candy_Database.gd` and, in the `actors_dictionary`, set any actor's "TTS" key to 1 to always enable TTS for that actor, or -1 to always disable it, overriding the default settings. If you don't want to override the default settings, set the "TTS" keys to 0.

For lines, in the Candy Dialogue Creator, you can toggle any spoken line's "TTS" button to indicate whether the line should comply with the default settings, or should override the defaults and the actor settings and either always use TTS or never use it.

func tts()

The `tts()` function is called by Candy DE to initiate TTS for spoken lines. It is called when spoken lines are processed (provided that TTS is enabled).

The `tts()` function is where the previously-mentioned TTS modes, such as "Simple" and "Advanced" are programmed: inside the function's match statement.

For each mode, you will need to write code to query a TTS tool, which will create an audio output.

You must then save that output -- either an `AudioStream` or a path to a voice file -- in the 'output' variable in the function.

Each mode will then return the value of the 'output' variable to Candy DE, which will play the resulting audio as it displays the line on screen.